

Entity Framework Core Database First Approach

In this article, I will discuss the **Entity Framework Core (EF Core) Database First Approach** with Examples. Please read our previous article discussing [Global Query Filters in Entity Framework Core](#) with Examples.

Entity Framework Core Database First Approach

The Database First approach in Entity Framework Core (EF Core) is a methodology where the data access layer (comprising models and a DbContext class) is generated from an existing database. This strategy is particularly useful when integrating EF Core with a pre-existing database or when database schema design is managed by a database administrator or an external team. So, this approach is particularly useful in the following scenarios:

- **Working with an existing database:** When the database already exists and is managed by database administrators or external teams.
- **Database-centric development:** When the database schema design is finalized before the application development begins.

With the Database First approach, EF Core will generate the necessary models, relationships, and DbContext class based on the structure of the database, enabling developers to access the database and perform database CRUD operations. This is often used in scenarios where the database schema has already been defined, and you want to avoid manually creating models and contexts.

Entity Framework Core Database First Approach with an E-commerce Application

Let us understand how to use the Entity Framework Core (EF Core) Database First approach step by step with an existing E-commerce database. This example will cover:

- **Creating the Database Schema:** Designing the tables and their relationships.
- **Scaffolding the DbContext:** Generating the DbContext and model classes from the database.
- **Performing CRUD Operations:** Creating, reading, updating, and deleting data.
- **Utilizing Views, Stored Procedures, and Functions:** Enhancing data operations with SQL Server features.

Key Entities in the E-commerce Database

The E-commerce database will manage several key entities essential for an online shopping platform. The key tables and their purposes are:

- **Customers:** Stores personal information of customers.
- **Addresses:** Holds multiple addresses associated with each customer.
- **Categories:** Organizes products into various classifications.
- **Products:** Contains detailed information about each product.
- **Orders:** Records customer orders.
- **OrderItems:** Lists the products included in each order.
- **Payments:** Tracks payment details for orders.

SQL Scripts for Database Creation

Let's start by creating the EcommerceDB database and all necessary tables with proper relationships, views, stored procedures, and functions. So, please execute the following SQL script in SQL Server Management Studio (SSMS).

```
CREATE DATABASE EcommerceDB;  
GO
```

```

USE EcommerceDB;
GO
-- Create Customers Table
-- Holds customer details like name, email, phone, and date of birth.
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY IDENTITY,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100) NOT NULL UNIQUE,
    Phone NVARCHAR(20),
    DateOfBirth DATE,
    CreatedDate DATETIME DEFAULT GETDATE()
);
GO
-- Create Addresses Table
-- Stores multiple addresses per customer, including whether an address is
the default.
CREATE TABLE Addresses (
    AddressID INT PRIMARY KEY IDENTITY,
    CustomerID INT NOT NULL,
    AddressLine1 NVARCHAR(100) NOT NULL,
    AddressLine2 NVARCHAR(100),
    City NVARCHAR(50) NOT NULL,
    State NVARCHAR(50) NOT NULL,
    PostalCode NVARCHAR(20) NOT NULL,
    Country NVARCHAR(50) NOT NULL,
    IsDefault BIT DEFAULT 0,
    CreatedDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
GO
-- Create Categories Table
-- Organizes products into categories, supporting hierarchical relationships
with ParentCategoryID.
CREATE TABLE Categories (
    CategoryID INT PRIMARY KEY IDENTITY,
    CategoryName NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500),
    ParentCategoryID INT NULL,
    CreatedDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (ParentCategoryID) REFERENCES Categories(CategoryID)
);
GO
-- Create Products Table
-- Contains product information, pricing, stock levels, and category
association.
CREATE TABLE Products (
    ProductID INT PRIMARY KEY IDENTITY,
    ProductName NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500),
    Price DECIMAL(18, 2) NOT NULL,

```

```

    CategoryID INT NOT NULL,
    StockQuantity INT NOT NULL,
    CreatedDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);
GO
-- Create Orders Table
-- Records each order's details, including the customer, shipping address,
total amount, and status.
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY IDENTITY,
    CustomerID INT NOT NULL,
    OrderDate DATETIME DEFAULT GETDATE(),
    ShippingAddressID INT NOT NULL,
    TotalAmount DECIMAL(18,2),
    Status NVARCHAR(50),
    CreatedDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
    FOREIGN KEY (ShippingAddressID) REFERENCES Addresses(AddressID)
);
GO
-- Create OrderItems Table
-- Lists individual products within an order, their quantities, and pricing.
CREATE TABLE OrderItems (
    OrderItemID INT PRIMARY KEY IDENTITY,
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(18,2) NOT NULL,
    TotalPrice DECIMAL(18,2) NOT NULL,
    CreatedDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
GO
-- Create Payments Table
-- Tracks payment information related to orders, such as payment method and
transaction status.
CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY IDENTITY,
    OrderID INT NOT NULL,
    PaymentDate DATETIME DEFAULT GETDATE(),
    Amount DECIMAL(18,2) NOT NULL,
    PaymentMethod NVARCHAR(50) NOT NULL,
    TransactionID NVARCHAR(100),
    Status NVARCHAR(50),
    CreatedDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
GO
-- View to Display Order Details

```

```

-- This view provides detailed information about orders, including customer
details, order items, and shipping addresses.
CREATE VIEW OrderDetailsView AS
SELECT
    o.OrderID,
    o.OrderDate,
    o.TotalAmount,
    o.Status AS OrderStatus,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    ca.AddressLine1 + ', ' + ca.City + ', ' + ca.State + ', ' + ca.Country AS
ShippingAddress,
    p.ProductName,
    oi.Quantity,
    oi.UnitPrice,
    oi.TotalPrice
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID
INNER JOIN Addresses ca ON o.ShippingAddressID = ca.AddressID
INNER JOIN OrderItems oi ON o.OrderID = oi.OrderID
INNER JOIN Products p ON oi.ProductID = p.ProductID;
GO

-- Stored Procedure to Get Orders by CustomerID
-- This stored procedure retrieves all orders placed by a specific customer.
CREATE OR ALTER PROCEDURE [dbo].[GetOrdersByCustomerID]
    @CustomerID INT
AS
BEGIN
    SELECT
        o.OrderID,
        o.OrderDate,
        o.TotalAmount,
        o.Status,
        o.CreatedDate,
        o.CustomerID,
        o.ShippingAddressId
    FROM Orders o
    INNER JOIN Customers c ON o.CustomerID = c.CustomerID
    WHERE o.CustomerID = @CustomerID;
END;
GO

-- Function to Calculate Discount
-- This function calculates discounts based on the total amount of an order.
CREATE FUNCTION CalculateDiscount
(
    @TotalAmount DECIMAL(18,2)
)
RETURNS DECIMAL(18,2)
AS
BEGIN
    DECLARE @Discount DECIMAL(18,2);
    IF @TotalAmount > 100

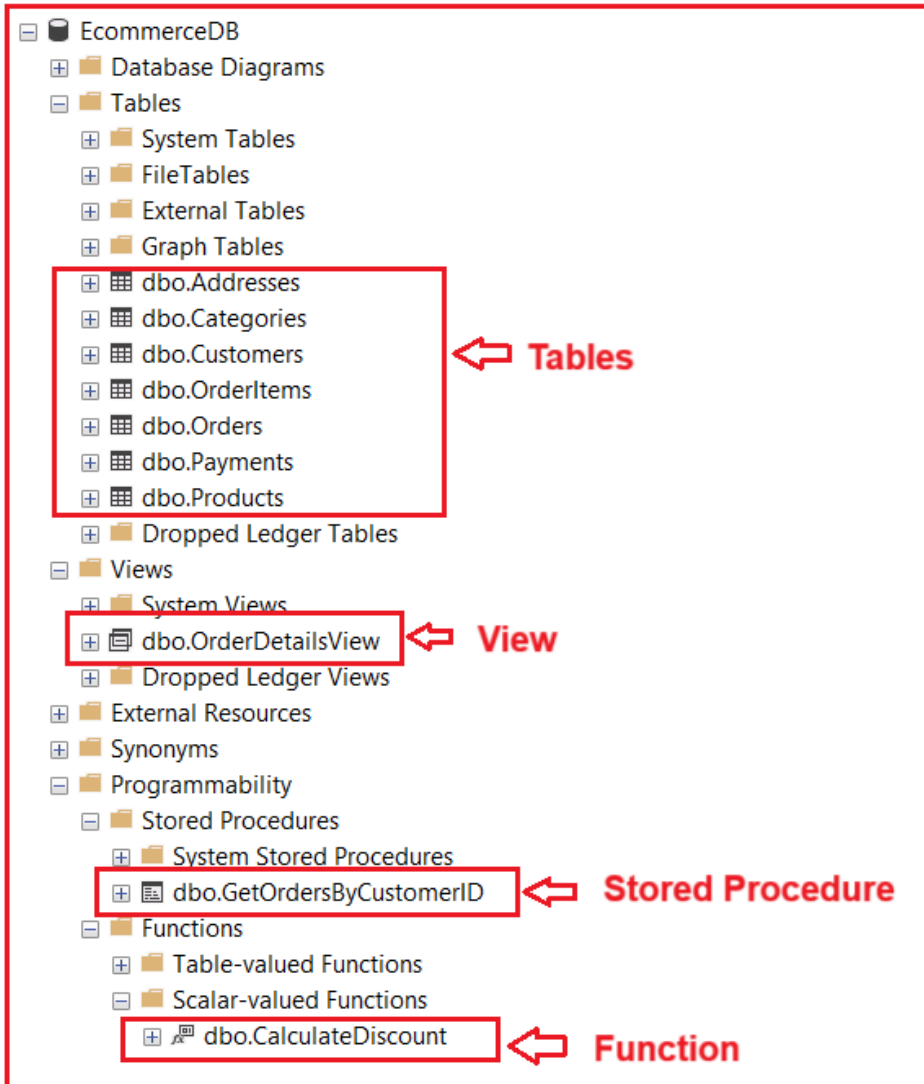
```

```

        SET @Discount = @TotalAmount * 0.10;
ELSE
    SET @Discount = 0;
RETURN @Discount;
END;
GO

```

Once you execute the above SQL Script, the database with the required database objects is created, as shown in the below image:



Creating a New Console Application:

We can use the EF Core Database First Approach with any type of Dot Net Core Application, including ASP.NET Core MVC, ASP.NET Core Web API, Console Application, etc. So, let us create a new Console Application named **ECommerceApp**.

Once you create the Console Application, we need to add the **Microsoft.EntityFrameworkCore.SqlServer** (EF Core provider for SQL Server) and **Microsoft.EntityFrameworkCore.Tools** (Tools for EF Core, including the Scaffold-DbContext command) packages. Please execute the following commands in the Package Manager Console to install these two packages.

- **Install-Package Microsoft.EntityFrameworkCore.SqlServer**

- [Install-Package Microsoft.EntityFrameworkCore.Tools](#)

Implementing EF Core Database First Approach

Now, we will see how to create the Context and Entity classes from our existing EcommerceDB database using the Entity Framework Core Database First Approach. Creating Context and Entity classes for an existing database is called Database-First Approach.

Entity Framework Core does not support Visual Designer for DB model and wizard to create entity and context classes similar to Entity Framework 6. So, we need to use the **Scaffold-DbContext** command, which is also called Reverse Engineering.

The **Scaffold-DbContext** command creates entities and context classes based on the schema of the existing database. We need to do this using **Package Manager Console** (PMC) tools.

Scaffolding the Database in EF Core

Scaffolding is the process of generating the DbContext and entity classes based on the existing database schema. Let us first understand the syntax of the Scaffold-DbContext

Syntax: [Scaffold-DbContext](#) ["Connection String"](#) [Microsoft.EntityFrameworkCore.SqlServer](#) **-o Models -f**

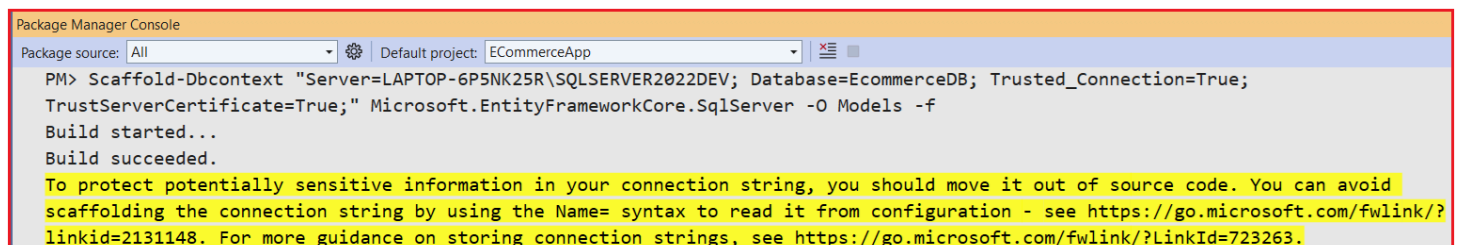
Parameters:

- **Connection String:** Specifies how to connect to the database.
- **Microsoft.EntityFrameworkCore.SqlServer:** Specifies the EF Core provider.
- **-o Models:** Output directory for the generated classes.
- **-f:** Forces scaffolding by overwriting existing files.

Example: Connecting to SQL Server EcommerceDB database

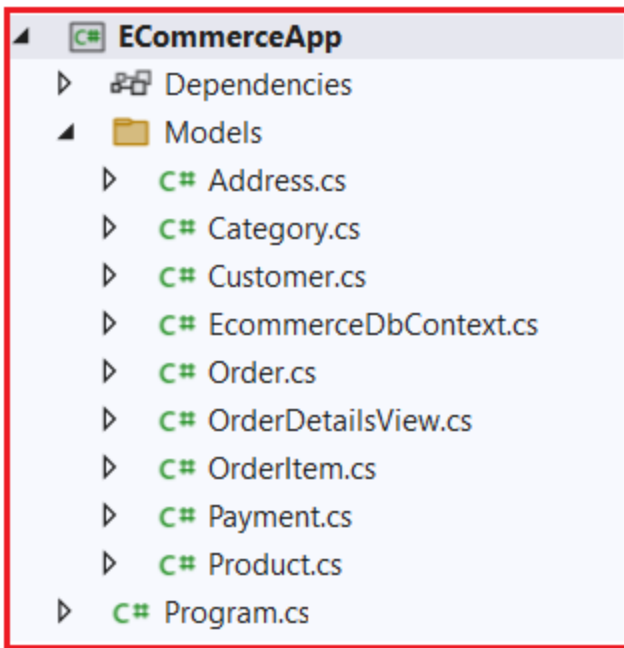
[Scaffold-DbContext](#) ["Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV; Database=EcommerceDB; Trusted_Connection=True; TrustServerCertificate=True;"](#) [Microsoft.EntityFrameworkCore.SqlServer](#) **-O Models -f**

So, open the Package Manager Console and execute the following command to create the entities and context class based on the EcommerceDB database:



```
Package Manager Console
Package source: All
Default project: ECommerceApp
PM> Scaffold-DbContext "Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV; Database=EcommerceDB; Trusted_Connection=True; TrustServerCertificate=True;" Microsoft.EntityFrameworkCore.SqlServer -O Models -f
Build started...
Build succeeded.
To protect potentially sensitive information in your connection string, you should move it out of source code. You can avoid scaffolding the connection string by using the Name= syntax to read it from configuration - see https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on storing connection strings, see https://go.microsoft.com/fwlink/?LinkId=723263.
```

Once you execute the above command, it should create the Models folder. It should have created the DbContext class and the required entities inside that folder, as shown in the image below.



Here, you can see that entities are created for all database tables and views, and the EcommerceDbContext class manages the database operations using these entities.

Performing CRUD Operations using EF Core DB First Approach

Now, let us Proceed and see how we can perform database CRUD Operations using the Entity Framework Core Database First Approach. Later, I will show how to use Views, Stored Procedures, and Stored Functions.

Adding Data

Let us see how to add new categories, products, customers, addresses, orders, order items, and payments. To better understand, please modify the Program class as follows. The following example code is self-explained, so please read the comment lines.

```
using ECommerceApp.Models;
using Microsoft.EntityFrameworkCore;
namespace ECommerceApp
{
    public class Program
    {
        public static void Main(string[] args)
        {
            using var context = new EcommerceDbContext();
            var dbTransaction = context.Database.BeginTransaction();
            try
            {
                // *** Create Operations ***
                // Adding Categories
                Console.WriteLine("Adding new categories...");
                // Create a new 'Electronics' category
                var electronicsCategory = new Category
                {
                    CategoryName = "Electronics",
```

```

        Description = "Electronic devices and gadgets",
        CreatedDate = DateTime.Now
    };
    context.Categories.Add(electronicsCategory);
    // Create a new 'Clothing' category
    var clothingCategory = new Category
    {
        CategoryName = "Clothing",
        Description = "Men's and Women's Clothing",
        CreatedDate = DateTime.Now
    };
    context.Categories.Add(clothingCategory);
    // Save categories to the database
    context.SaveChanges();
    Console.WriteLine("Categories 'Electronics' and 'Clothing'
added successfully.\n");
    // Adding Products
    Console.WriteLine("Adding new products...");
    // Create a new product 'Smartphone' under 'Electronics'
category
    var product1 = new Product
    {
        ProductName = "Smartphone",
        Description = "Latest model smartphone",
        Price = 699.99M,
        StockQuantity = 50,
        CategoryId = electronicsCategory.CategoryId,
        CreatedDate = DateTime.Now
    };
    context.Products.Add(product1);
    // Create a new product 'Jeans' under 'Clothing' category
    var product2 = new Product
    {
        ProductName = "Jeans",
        Description = "Blue denim jeans",
        Price = 49.99M,
        StockQuantity = 100,
        CategoryId = clothingCategory.CategoryId,
        CreatedDate = DateTime.Now
    };
    context.Products.Add(product2);
    // Save products to the database
    context.SaveChanges();
    Console.WriteLine("Products 'Smartphone' and 'Jeans' added
successfully.\n");
    // Adding a Customer
    Console.WriteLine("Adding a new customer...");
    // Create a new customer 'John Doe'
    var customer = new Customer
    {
        FirstName = "John",

```

```

        LastName = "Doe",
        Email = "john.doe@example.com",
        Phone = "1234567890",
        DateOfBirth = new DateOnly(1990, 1, 1),
        CreatedDate = DateTime.Now
    };
    context.Customers.Add(customer);
    // Save customer to the database
    context.SaveChanges();
    Console.WriteLine($"Customer '{customer.FirstName}
{customer.LastName}' added successfully with CustomerID:
{customer.CustomerId}\n");
    // Adding Customer Address
    Console.WriteLine("Adding address for the customer...");
    // Create a new address for the customer
    var address1 = new Address
    {
        CustomerId = customer.CustomerId,
        AddressLine1 = "123 Main Street",
        AddressLine2 = null,
        City = "Anytown",
        State = "Anystate",
        PostalCode = "12345",
        Country = "USA",
        IsDefault = false,
        CreatedDate = DateTime.Now
    };
    context.Addresses.Add(address1);
    var address2 = new Address
    {
        CustomerId = customer.CustomerId,
        AddressLine1 = "123 Main Street",
        AddressLine2 = null,
        City = "Anytown",
        State = "Anystate",
        PostalCode = "12345",
        Country = "USA",
        IsDefault = true,
        CreatedDate = DateTime.Now
    };
    context.Addresses.Add(address2);
    // Save address to the database
    context.SaveChanges();
    Console.WriteLine("Customer addresses added
successfully.\n");
    // Creating an Order
    Console.WriteLine("Creating a new order for the
customer...");
    // Create a new order for the customer with the shipping
    address
    var order = new Order

```

```

    {
        CustomerId = customer.CustomerId,
        ShippingAddressId = address2.AddressId,
        TotalAmount = product1.Price, // We are setting Total
amount is the price of the product. But we need to calculate the Total Amount
        Status = "Pending",
        CreatedDate = DateTime.Now
    };
    context.Orders.Add(order);
    // Save order to the database
    context.SaveChanges();
    Console.WriteLine($"Order created successfully with OrderID:
{order.OrderId}\n");
    // Adding Order Item
    Console.WriteLine("Adding order item to the order...");
    // Create a new order item for the order
    var orderItem = new OrderItem
    {
        OrderId = order.OrderId,
        ProductId = product1.ProductId,
        Quantity = 1,
        UnitPrice = product1.Price,
        TotalPrice = product1.Price * 1,
        CreatedDate = DateTime.Now
    };
    orderItem.TotalPrice = orderItem.UnitPrice *
orderItem.Quantity;
    context.OrderItems.Add(orderItem);
    // Save order item to the database
    context.SaveChanges();
    Console.WriteLine("Order item added successfully.\n");
    // Processing Payment
    Console.WriteLine("Processing payment for the order...");
    // Create a new payment for the order
    var payment = new Payment
    {
        OrderId = order.OrderId,
        Amount = Convert.ToDecimal(order.TotalAmount),
        PaymentMethod = "Credit Card",
        TransactionId = "TXN123456",
        Status = "Pending",
        CreatedDate = DateTime.Now
    };
    context.Payments.Add(payment);
    // Save payment to the database
    context.SaveChanges();
    dbTransction.Commit();
    Console.WriteLine("Payment processed successfully.\n");
}
catch (DbUpdateException ex)
{

```

```

        dbTransction.Rollback();
        Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
    }
    catch (Exception ex)
    {
        dbTransction.Rollback();
        // Display any errors that occur during the operations
        Console.WriteLine($"An error occurred: {ex.Message}");
    }
}
}
}
}
}

```

Output:

```

Adding new categories...
Categories 'Electronics' and 'Clothing' added successfully.

Adding new products...
Products 'Smartphone' and 'Jeans' added successfully.

Adding a new customer...
Customer 'John Doe' added successfully with CustomerID: 1

Adding address for the customer...
Customer address added successfully.

Creating a new order for the customer...
Order created successfully with OrderID: 1

Adding order item to the order...
Order item added successfully.

Processing payment for the order...
Payment processed successfully.

```

Reading Data:

Let us retrieve and display orders with the customer, order item, and payment details. To better understand, please modify the Program class as follows. The following example code is self-explained, so please read the comment lines.

```

using ECommerceApp.Models;
using Microsoft.EntityFrameworkCore;
namespace EcommerceApp
{
    public class Program
    {
        public static void Main(string[] args)
        {
            try

```

```

{
    // Create an instance of the DbContext to interact with the
database
    using var context = new EcommerceDbContext();
    // *** Retrieve and Display Orders with Details ***
    Console.WriteLine("Fetching and displaying all orders with
customer, order items, and payment details...\n");
    // Fetch orders including related data:
    var orders = context.Orders
        .Include(o => o.Customer) // Include customer information
        .Include(o => o.OrderItems) // Include order items
        .ThenInclude(oi => oi.Product) // Include product
details for each order item
        .Include(o => o.Payments) // Include payments associated
with the order
        .Include(o => o.ShippingAddress) // Include shipping
address
        .ToList();
    // Check if any orders exist
    if (orders.Any())
    {
        // Iterate through each order
        foreach (var order in orders)
        {
            // Display basic order information
            Console.WriteLine($"Order ID: {order.OrderId}, Date:
{order.OrderDate}, Status: {order.Status}, Total Amount:
{order.TotalAmount}");

            // Display customer information
            Console.WriteLine($"Customer:
{order.Customer.FirstName} {order.Customer.LastName}");
            Console.WriteLine($"Email: {order.Customer.Email}");
            Console.WriteLine($"Phone: {order.Customer.Phone}");
            // Display shipping address
            var address = order.ShippingAddress;
            Console.WriteLine("\nShipping Address:");
            Console.WriteLine($"  {address.AddressLine1}");
            if (!string.IsNullOrEmpty(address.AddressLine2))
            {
                Console.WriteLine($"  {address.AddressLine2}");
            }
            Console.WriteLine($"  {address.City},
{address.State}, {address.PostalCode}, {address.Country}");
            // Display order items
            Console.WriteLine("\nOrder Items:");
            foreach (var item in order.OrderItems)
            {
                Console.WriteLine($"  Product:
{item.Product.ProductName}, Quantity: {item.Quantity}, Unit Price:
{item.UnitPrice}, Total Price: {item.TotalPrice}");
            }
        }
    }
}

```

```

        Console.WriteLine(); // Blank line for
readability
    }
    // Display payment details
    Console.WriteLine("Payments:");
    foreach (var payment in order.Payments)
    {
        Console.WriteLine($"Payment ID:
{payment.PaymentId}, Amount: {payment.Amount}, Payment Method:
{payment.PaymentMethod}");
        Console.WriteLine($"Payment Date:
{payment.PaymentDate}, Transaction ID: {payment.TransactionId}, Payment
Status: {payment.Status}");
        Console.WriteLine(); // Blank line for
readability
    }
    Console.WriteLine(); // Separator for readability
}
else
{
    Console.WriteLine("No orders found in the database.");
}
}
catch (DbUpdateException ex)
{
    Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
}
catch (Exception ex)
{
    // Display any errors that occur during the operations
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}
}

```

Output:

Fetching and displaying all orders with customer, order items, and payment details...

Order ID: 1, Date: 26-09-2024 20:44:43, Status: Pending, Total Amount: 699.99

Customer: John Doe

Email: john.doe@example.com

Phone: 1234567890

Shipping Address:

123 Main Street

Anytown, Anystate, 12345, USA

Order Items:

Product: Smartphone, Quantity: 1, Unit Price: 699.99, Total Price: 699.99

Payments:

Payment ID: 1, Amount: 699.99, Payment Method: Credit Card

Payment Date: 26-09-2024 20:44:43, Transaction ID: TXN123456, Payment Status: Pending

Updating Data

Let us update the status of an order and payment and adjust the stock quantities of the products in the order. To better understand, please modify the Program class as follows. The following example code is self-explained, so please read the comment lines.

```
using ECommerceApp.Models;
using Microsoft.EntityFrameworkCore;
namespace ECommerceApp
{
    public class Program
    {
        public static void Main(string[] args)
        {
            using var context = new ECommerceDbContext();
            var dbTransaction = context.Database.BeginTransaction();
            try
            {
                // *** Update Order Status, Payment Status, and Adjust Stock
                Quantities ***
                // Specify the Order ID that you want to update
                int orderIdToUpdate = 1; // Change this to the actual Order
                ID
                Console.WriteLine($"Updating Order ID: {orderIdToUpdate}\n");
                // Retrieve the order including related entities:
                // - Include Order Items and their associated Products
                // - Include Payments associated with the order
                var order = context.Orders
                    .Include(o => o.OrderItems)
                    .ThenInclude(oi => oi.Product)
                    .Include(o => o.Payments)
                    .FirstOrDefault(o => o.OrderId == orderIdToUpdate);
                // Check if the order exists
                if (order != null)
                {
                    // Update payment status if payment exists
```

```

        //var payment = order.Payments.FirstOrDefault();
        foreach (var payment in order.Payments)
        {
            // Display current payment status
            Console.WriteLine($"Current Payment Status:
{payment.Status}");

            // Update the payment status to 'Completed'
            payment.Status = "Completed";
            // Display updated payment status
            Console.WriteLine($"Updated Payment Status:
{payment.Status}\n");
        }
        //Updating Order Status
        // Display current order status
        Console.WriteLine($"Current Order Status:
{order.Status}");

        // Update the order status to 'Processing'
        order.Status = "Processing";
        // Display updated order status
        Console.WriteLine($"Updated Order Status:
{order.Status}\n");

        // Adjust stock quantities for each product in the order
        Console.WriteLine("Adjusting stock quantities for ordered
products...");

        foreach (var orderItem in order.OrderItems)
        {
            var product = orderItem.Product;
            // Display current stock quantity
            Console.WriteLine($"Product: {product.ProductName},
Current Stock Quantity: {product.StockQuantity}");
            // Decrease the stock quantity by the quantity
            ordered
            orderItem.Quantity;

            product.StockQuantity = product.StockQuantity -
            orderItem.Quantity;

            // Ensure stock quantity does not go negative
            if (product.StockQuantity < 0)
            {
                product.StockQuantity = 0;
            }
            // Display updated stock quantity
            Console.WriteLine($"Updated Stock Quantity:
{product.StockQuantity}\n");
        }
        // Save all changes to the database
        context.SaveChanges();
        dbTransaction.Commit();
        Console.WriteLine("Order status, payment status, and
stock quantities updated successfully.\n");
    }
    else
    {

```

```

        Console.WriteLine($"Order with ID {orderIdToUpdate} not
found.\n");
    }
    // Display the updated order and product details for
confirmation
    Console.WriteLine("Fetching updated order and product
details...\n");
    // Retrieve the updated order
    var updatedOrder = context.Orders
        .Include(o => o.OrderItems)
        .ThenInclude(oi => oi.Product)
        .Include(o => o.Payments)
        .FirstOrDefault(o => o.OrderId == orderIdToUpdate);
    if (updatedOrder != null)
    {
        // Display order status
        Console.WriteLine($"Order ID: {updatedOrder.OrderId},
Order Status: {updatedOrder.Status}");
        // Display payment status
        var updatedPayment =
updatedOrder.Payments.FirstOrDefault();
        if (updatedPayment != null)
        {
            Console.WriteLine($"Payment Status:
{updatedPayment.Status}");
        }
        // Display updated stock quantities of products
        Console.WriteLine("\nUpdated Product Stock Quantities:");
        foreach (var orderItem in updatedOrder.OrderItems)
        {
            var product = orderItem.Product;
            Console.WriteLine($"Product: {product.ProductName},
Stock Quantity: {product.StockQuantity}");
        }
    }
    else
    {
        Console.WriteLine("Order not found after update.\n");
    }
}
catch (DbUpdateException ex)
{
    dbTransaction.Rollback();
    Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
}
catch (Exception ex)
{
    dbTransaction.Rollback();
    // Display any errors that occur during the operations
    Console.WriteLine($"An error occurred: {ex.Message}");
}

```

```

    }
}
}

```

Output:

```

Updating Order ID: 1

Current Order Status: Pending
Updated Order Status: Delivered

Current Payment Status: Pending
Updated Payment Status: Completed

Adjusting stock quantities for ordered products...
Product: Smartphone, Current Stock Quantity: 50
Updated Stock Quantity: 49

Order status, payment status, and stock quantities updated successfully.

Fetching updated order and product details...

Order ID: 1, Order Status: Delivered
Payment Status: Completed

Updated Product Stock Quantities:
Product: Smartphone, Stock Quantity: 49

```

Deleting Data

Let us delete an order and the associated Order Items and payments. To better understand, please modify the Program class as follows. The following example code is self-explained, so please read the comment lines.

```

using ECommerceApp.Models;
using Microsoft.EntityFrameworkCore;
namespace ECommerceApp
{
    public class Program
    {
        public static void Main(string[] args)
        {
            try
            {
                using var context = new EcommerceDbContext();
                // *** Delete an Order and Associated Order Items and
                Payments ***
                // Specify the Order ID that you want to delete
                int orderIdToDelete = 1; // Change this to the actual Order
                ID you want to delete
            }
            catch { }
        }
    }
}

```

```

        Console.WriteLine($"Attempting to delete Order ID:
{orderIdToDelete}\n");
        // Retrieve the order including related entities:
        // - Include Order Items
        // - Include Payments
        var order = context.Orders
            .Include(o => o.OrderItems)
            .Include(o => o.Payments)
            .FirstOrDefault(o => o.OrderId == orderIdToDelete);
        // Check if the order exists
        if (order != null)
        {
            // Display order details
            Console.WriteLine("Order Details:");
            Console.WriteLine($"Order ID: {order.OrderId}, Date:
{order.OrderDate}, Status: {order.Status}, Total Amount:
{order.TotalAmount}\n");
            // Display associated order items
            Console.WriteLine("Associated Order Items:");
            foreach (var orderItem in order.OrderItems)
            {
                Console.WriteLine($"Order Item ID:
{orderItem.OrderItemId}, Product ID: {orderItem.ProductId}, Quantity:
{orderItem.Quantity}, Total Price: {orderItem.TotalPrice}");
            }
            // Display associated payments
            Console.WriteLine("\nAssociated Payments:");
            foreach (var payment in order.Payments)
            {
                Console.WriteLine($"Payment ID:
{payment.PaymentId}, Amount: {payment.Amount}, Status: {payment.Status}");
            }
            Console.WriteLine("\nProceeding to delete the order and
its associated order items and payments...\n");
            // Remove order items first due to foreign key
constraints
            if (order.OrderItems.Any())
            {
                context.OrderItems.RemoveRange(order.OrderItems);
                Console.WriteLine("Order items deleted
successfully.");
            }
            // Remove payments associated with the order
            if (order.Payments.Any())
            {
                context.Payments.RemoveRange(order.Payments);
                Console.WriteLine("Payments deleted successfully.");
            }
            // Remove the order
            context.Orders.Remove(order);
            Console.WriteLine("Order deleted successfully.");

```

```

        // Save all changes to the database
        context.SaveChanges();
        Console.WriteLine("\nAll changes have been saved to the
database.");
        // Confirm Deletion
        Console.WriteLine("\nVerifying that the order and
associated records have been deleted...");
        // Attempt to retrieve the order again
        var deletedOrder = context.Orders
            .Include(o => o.OrderItems)
            .Include(o => o.Payments)
            .FirstOrDefault(o => o.OrderId == orderIdToDelete);
        if (deletedOrder == null)
        {
            Console.WriteLine("Order deletion confirmed. The
order and its associated order items and payments have been successfully
deleted.");
        }
        else
        {
            Console.WriteLine("Order deletion failed. The order
still exists in the database.");
        }
    }
    else
    {
        Console.WriteLine($"Order with ID {orderIdToDelete} not
found. No deletion performed.");
    }
}
catch (DbUpdateException ex)
{
    Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
}
catch (Exception ex)
{
    // Display any errors that occur during the operations
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}

```

Output:

```

Attempting to delete Order ID: 1

Order Details:
Order ID: 1, Date: 26-09-2024 20:44:43, Status: Delivered, Total Amount: 699.99

Associated Order Items:
    Order Item ID: 1, Product ID: 1, Quantity: 1, Total Price: 699.99

Associated Payments:
    Payment ID: 1, Amount: 699.99, Status: Completed

Proceeding to delete the order and its associated order items and payments...

Order items deleted successfully.
Payments deleted successfully.
Order deleted successfully.

All changes have been saved to the database.

Verifying that the order and associated records have been deleted...
Order deletion confirmed. The order and its associated order items and payments have been successfully deleted.

```

Stored Function using EF Core:

Even though we are using a Database-First approach, we need to manually define the function in our DbContext class, as EF Core does not automatically import functions through the scaffolding process. So, add the following method in EcommerceDbContext that represents the scalar function. Use the DbFunction attribute to link it to the SQL function.

```

// Define the stored function
[DbFunction("CalculateDiscount", "dbo")]
public static decimal? CalculateDiscount(decimal? totalAmount)
{
    // This method is for EF Core to know how to call the function.
    throw new NotImplementedException();
}

```

Once you have defined the function in the EcommerceDbContext class, you can call it like any other static method in your code.

Using Views, Stored Procedures, and Functions in EF Core DB First Approach:

Please modify the Program class as follows: Here, I am showing how to use Stored procedures and views with Entity Framework Core Database First Approach using the FromSqlRaw method. At the same time, we can call the CalculateDiscount method, which will execute the CalculateDiscount function. The following code is self-explained, so please read the comment lines for a better understanding.

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Data.SqlClient;
using ECommerceApp.Models;
namespace EcommerceApp
{
    public class Program
    {
        public static void Main(string[] args)
        {
            // Create an instance of the DbContext
            using var context = new EcommerceDbContext();
            var dbTransaction = context.Database.BeginTransaction();

```

```

    try
    {
        // *** Step 1: Create a New Order with Discount Applied ***
        Console.WriteLine("Creating a new order with discount
applied...\n");
        // Define the existing CustomerID and ProductID
        int existingCustomerId = 1; // Replace with actual CustomerID
        int existingProductId = 1;   // Replace with actual ProductID
        // Retrieve the existing customer and product from the
database
        var customer = context.Customers.FirstOrDefault(c =>
c.CustomerId == existingCustomerId);
        var product1 = context.Products.FirstOrDefault(p =>
p.ProductId == existingProductId);
        var product2 = context.Products.FirstOrDefault(p =>
p.ProductId == 2);
        if (customer == null || product1 == null || product2 == null)
        {
            Console.WriteLine("Customer or Product not found. Cannot
proceed with order creation.");
            return;
        }
        // Retrieve the customer's default shipping address
        var shippingAddress = context.Addresses
            .FirstOrDefault(a => a.CustomerId == customer.CustomerId
&& a.IsDefault == true);
        if (shippingAddress == null)
        {
            Console.WriteLine("Shipping address not found for the
customer.");
            return;
        }
        // Create a new order for the customer
        var newOrder = new Order
        {
            CustomerId = customer.CustomerId,
            ShippingAddressId = shippingAddress.AddressId,
            Status = "Pending",
            CreatedDate = DateTime.Now
        };
        context.Orders.Add(newOrder);
        try
        {
            context.SaveChanges(); // Save to get the OrderID
            Console.WriteLine($"Order created successfully with
OrderID: {newOrder.OrderId}\n");
            // Add an order item to the new order
            int quantityOrdered = 2; // Assuming the customer orders
2 units
            List<OrderItem> orderItems = new List<OrderItem>()
            {

```

```

        new OrderItem
        {
            OrderId = newOrder.OrderId,
            ProductId = product1.ProductId,
            Quantity = quantityOrdered,
            UnitPrice = product1.Price,
            TotalPrice = product1.Price * quantityOrdered,
            CreatedDate = DateTime.Now
        },
        new OrderItem
        {
            OrderId = newOrder.OrderId,
            ProductId = product2.ProductId,
            Quantity = 1,
            UnitPrice = product2.Price,
            TotalPrice = product2.Price * 1,
            CreatedDate = DateTime.Now
        }
    };
    context.OrderItems.AddRange(orderItems);
    // Update the order's total amount before discount
    decimal totalAmountBeforeDiscount = orderItems.Sum(ord =>
ord.TotalPrice);
    // Apply the discount using the stored function
    'CalculateDiscount'
        decimal? discount = context.Orders
            .Where(o => o.OrderId == newOrder.OrderId)
            .Select(o =>
EcommerceDbContext.CalculateDiscount(totalAmountBeforeDiscount))
            .FirstOrDefault();
    // Ensure discount is not null
    discount ??= 0;
    // Calculate the total amount after applying the discount
    decimal totalAmountAfterDiscount =
totalAmountBeforeDiscount - discount.Value;
    // Update the order's total amount
    newOrder.TotalAmount = totalAmountAfterDiscount;
    // Save changes to persist updates
    context.SaveChanges();
    Console.WriteLine($"Order item added, discount applied,
and order total amount updated.\n");
    // Display discount details
    Console.WriteLine($"Amount Before Discount:
{totalAmountBeforeDiscount}, Discount Applied: {discount.Value}, Amount After
Discount: {totalAmountAfterDiscount}\n");
    // Process payment for the order
    var payment = new Payment
    {
        OrderId = newOrder.OrderId,
        Amount = Convert.ToDecimal(newOrder.TotalAmount), //
Amount after discount

```

```

        PaymentMethod = "Credit Card",
        TransactionId = "TXN" + DateTime.Now.Ticks,
        Status = "Processing",
        Discount = discount,
        CreatedDate = DateTime.Now
    };
    context.Payments.Add(payment);
    context.SaveChanges();
    Console.WriteLine("Payment processed for the new order
with discounted amount.\n");
    dbTransaction.Commit();
    // *** Step 2: Use the View 'OrderDetailsView' ***
}
catch (Exception)
{
    dbTransaction.Rollback();
}

Console.WriteLine("Fetching order details using the
'OrderDetailsView'...\n");
// Retrieve order details for the new order
var orderDetails = context.OrderDetailsViews
    .Where(od => od.OrderId == newOrder.OrderId)
    .ToList();
if (orderDetails.Any())
{
    foreach (var detail in orderDetails)
    {
        Console.WriteLine($"Order ID: {detail.OrderId}, Date:
{detail.OrderDate}, Status: {detail.OrderStatus}");
        Console.WriteLine($"Customer Name:
{detail.CustomerName}, Shipping Address: {detail.ShippingAddress}");
        Console.WriteLine($"Product Name:
{detail.ProductName}, Quantity: {detail.Quantity}, Unit Price:
{detail.UnitPrice}, Total Price: {detail.TotalPrice} \n");
    }
}
else
{
    Console.WriteLine("No order details found for the new
order.");
}
// *** Step 3: Use the Stored Procedure
'GetOrdersByCustomerID' ***
Console.WriteLine("\nFetching orders for the customer using
the stored procedure 'GetOrdersByCustomerID'...\n");
// Define the parameter
var customerIdParam = new SqlParameter("@CustomerId",
customer.CustomerId);
// Execute the stored procedure
var orders = context.Orders

```

```

        .FromSqlRaw("EXEC GetOrdersByCustomerID @CustomerID",
customerIdParam)
        .ToList();
    if (orders.Any())
    {
        foreach (var order in orders)
        {
            Console.WriteLine($"Order ID: {order.OrderId}, Date:
{order.OrderDate}, Status: {order.Status}, Total Amount:
{order.TotalAmount}");
        }
    }
    else
    {
        Console.WriteLine("No orders found for the customer.");
    }
}
catch (DbUpdateException ex)
{
    Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
}
catch (Exception ex)
{
    // Display any errors that occur during the operations
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}

```

We have successfully implemented how to use the EF Core Database First approach with an E-commerce database. We generated the required models by reverse-engineering the existing database and performing CRUD operations. Additionally, we use database views, stored procedures, and functions within our application.

Note: If the database schema changes, the models and DbContext will need to be updated using the scaffold command with the -Force option to regenerate the models, which will overwrite any customizations.

In this article, I explain the [CRUD Operations in ASP.NET Core MVC using Entity Framework Core](#) with Examples. I hope you enjoy this article on **the Entity Framework Core Database First Approach**.